Лабораторна робота №11-12. REST API і TypeScript: Fetch API, async/await, типізація API-відповідей, union types, narrowing і type guards, loading та error states

Термін здачі: 19.10.2026

ЛР-11 і ЛР-12 об'єднано в одну роботу — ЛР-11-12. Попередній варіант ЛР-11 (ShowFinder на JavaScript з `check.html`) скасовано: здавати його не потрібно.

1. Мета роботи

Навчитися інтегрувати REST API у фронтенд-застосунок і типізувати цю інтеграцію в TypeScript: описувати відповіді API типами за документацією, писати типізовані запити з `fetch` і `async/await`, перетворювати дані з мережі з `unknown` на перевірені типи й показувати користувачеві стани завантаження, порожнього результату та помилки. Навчитися перевіряти код, написаний AI-агентом, компілятором, тестами й панеллю Network.

Очікуваний результат: застосунок ShowFinder на TypeScript працює зі справжнім TVmaze API; `npm run check` показує «Типи: без помилок» і всі тести пройдено; типи, запити й стани ви написали власноруч, а чернетку AI-агента перевірили й виправили.

2. Цілі та результати навчання

Після виконання лабораторної роботи студент повинен:

- Надсилати запити через `fetch`, читати `ok`, `status`, заголовки й тіло відповіді; пояснювати, чому `fetch` не відхиляє проміс на 404 і 500 і коли відхиляє.
- Писати асинхронний код з `async/await` і `try/catch` так, щоб жодна помилка не ставала `Uncaught (in promise)`.
- Пояснювати статуси 200, 404, 429 і 5xx, заголовки `Content-Type`, `Cache-Control` і `Access-Control-Allow-Origin` та що саме блокує CORS.

- Досліджувати запити в панелі Network: статус, заголовки, Payload, Preview, throttling.
- Описувати відповіді API інтерфейсами TypeScript за документацією й справжніми відповідями, з `null` там, де він буває.
- Типізувати функції й класи; розрізняти `unknown` і `any`; звужувати типи (`typeof`, `instanceof`, `in`, перевірка поля-дискримінанта) і писати type guards.
- Моделювати стани інтерфейсу discriminated union і обробляти їх вичерпно, з `never`.
- Пояснювати, що робить `tsc`, що вмикає `strict` і чому типи зникають після компіляції.
- Перевіряти код, написаний AI-агентом, компілятором, тестами й у браузері.

3. Завдання лабораторної роботи

1. Опрацювати дослідницькі завдання (§4): E1–E3 — у браузері, E4–E6 — у проєкті, коли дійдете до T1–T2.
2. Завантажити starter-проєкт, зафіксувати його в Git без змін і запустити `npm run check` (§5.1).
3. Виконати завдання T3, §4, по черзі: T1–T3 власноруч, T4 — з AI-агентом (§5.2).
4. Пройти чек-лист Network (T3, §6) і оформити звіт (§6). Потім — захист (§7).

4. Дослідницькі завдання

Довідка: Using the Fetch API, HTTP response status codes, CORS — MDN; Everyday Types, Narrowing, Object Types — TypeScript Handbook; Network features reference — Chrome DevTools; документація TVmaze API.

Протокол експерименту:

1. Прочитайте опис і код і запишіть, що очікуєте, — це «Мій прогноз».
2. Виконайте експеримент.
3. Якщо прогноз не збігся з результатом, поясніть, де саме він був хибним.

Результати зведіть у таблицю — вона входить у звіт:

#	Що досліджено	Мій прогноз	Фактичний результат	Пояснення (якщо не збіглося)
E1	відповідь API пошуку	…	…	…

4.1. HTTP і Fetch API — у браузері

Розпакуйте starter-проєкт (§5.1) і відкрийте `index.html` через Live Preview у Chrome або Edge за адресою `http://127.0.0.1:…`, а не у вбудованій вкладці VS Code. Фрагменти

вставляйте в Console цієї сторінки: Console дозволяє `await` без `async`-функції. У Network виберіть фільтр **Fetch/XHR**.

E1. Відповідь API.

```
const response = await fetch('https://api.tvmaze.com/search/shows?q=girls');
console.log(response.status, response.ok, response.headers.get('content-
type'));
const data = await response.json();
console.log(Array.isArray(data), data.length, Object.keys(data[0]));
```

Проаналізуйте: що виведе кожен `console.log`? Де в елементі відповіді назва серіалу? Знайдіть у Network → Response Headers заголовки `cache-control` і `access-control-allow-origin`: що кожен означає? Відкрийте ту саму адресу з `?q=kitchen` у новій вкладці: у якого серіалу немає постера чи дати прем'єри і як це виглядає в JSON?

E2. Коли fetch відхиляє проміс. Домен `.invalid` гарантовано не існує.

```
const missing = await fetch('https://api.tvmaze.com/shows/999999999');
console.log(missing.ok, missing.status);
console.log(await missing.json());
try {
  await fetch('https://api.tvmaze.invalid/shows/1');
} catch (error) {
  console.log('catch:', error.name, error.message);
}
```

Проаналізуйте: чи відхилить `fetch` проміс на 404? Що виведуть перші два `console.log`? Що потрапить у `catch` і чому саме туди? Що з цього має робити `getJson` (Т3, Т2.2)?

E3. CORS. `api.tvmaze.com` — API, `www.tvmaze.com` — сайт для людей.

```
const api = await fetch('https://api.tvmaze.com/shows/82');
console.log('api:', api.status);
try {
  const site = await fetch('https://www.tvmaze.com/shows/82');
  console.log('www:', site.status);
} catch (error) {
  console.log('www:', error.name, error.message);
}
```

Проаналізуйте: що виведе кожен рядок? Чи дійшов запит до `www.tvmaze.com` і чи прийшла відповідь — що каже Network? Чому ця сама адреса відкривається у вкладці, а `fetch` зі сторінки — ні? Хто тут дозволяє доступ і хто його забороняє?

4.2. TypeScript — у проєкті

E4. unknown, any і звуження (після T2). Наведіть курсор у редакторі на `response.json()` у `getJson`, потім у `searchShows` на дані, які повернула `getJson`: одразу після `await` і там, де ви їх уже перевірили guard-ом. Тимчасово допишіть одразу після `await` рядок `console.log(data.length);` (з вашою назвою змінної), подивіться на помилку й видаліть рядок.

Проаналізуйте: який тип має кожен із трьох виразів? Чому `getJson` оголошено як `Promise<unknown>`, а не `Promise<any>`? Що змінилося б у `searchShows`, якби вона повертала `any`?

E5. Режим strict (після T1). Створіть тимчасовий файл `src/strict.ts`:

```
import type { Show } from './types.ts';

export function posterUrl(show: Show) {
  return show.image.medium;
}

export function shout(text) {
  return text.toUpperCase();
}
```

Запустіть `npm run typecheck`. Потім у `tsconfig.json` тимчасово замініть `"strict": true` на `"strict": false` і запустіть ще раз.

Проаналізуйте: які помилки в `src/strict.ts` покаже компілятор у кожному випадку? Що зробив би `posterUrl` для серіалу Harlem's Kitchen (id 45962) під час виконання? Без `strict` з'являється нова помилка в `check/types.check.ts`: яке правило T1 без `strict` перевірити неможливо і чому? Поверніть `"strict": true` і видаліть `src/strict.ts`.

E6. Що лишається після компіляції (після T2). Поки працює `npm run watch`, відкрийте `dist/types.js` і `dist/api.js`.

Проаналізуйте: що лишилося від `interface Show`, від `value is Show` і від `readonly status: number`? Чому `isShow` не можна написати як `value instanceof Show`? Навіщо перевіряти форму відповіді під час виконання (T3, T2.7), якщо типи вже є?

5. Практична частина: як працювати

Starter-проєкт **ShowFinder** — пошук серіалів у TVmaze на TypeScript. Розмітка, стилі, показ даних і пошук елементів сторінки вже готові. Порожні чотири файли: `src/types.ts`, `src/api.ts`, `src/state.ts` і `src/app.ts`.

Усі завдання практичної частини — у ТЗ, §4: Т1–Т4, кожне з пронумерованими правилами, а в Т4 — ще й запит для AI-агента. Тримайте ТЗ відкритим під час роботи. Цей розділ — про те, як працювати: підготовка, правила роботи з AI-агентом і коміти.

Як перевіряти. `npm run check` спершу перевіряє типи (`tsc`), потім запускає тести (`node --test`) і закінчує підсумком «Типи: … · Тести: пройдено N з M». Назва кожного тесту й коментар біля кожної перевірки типів у `check/types.check.ts` починаються з номера правила ТЗ. Тести не ходять у мережу: вони підміняють `fetch` і відповідають записаними відповідями TVmaze, тож не залежать ні від Wi-Fi, ні від ліміту запитів. Справжній TVmaze і браузер перевіряє чек-лист Network (ТЗ, §6).

5.1. Підготовка

1. Перевірте `node -v`: потрібен Node.js 22.18 або новіший. На старішому тести пишуть `Unknown file extension ".ts"`.
2. Завантажте showfinder-ts-starter.zip і розпакуйте в **окремий репозиторій** або в теку `lab-11/` спільного репозиторію з лабораторними. Зробіть перший коміт **без змін**: `Starter`.
3. Виконайте `npm install`, потім `npm run watch` — і лишіть його працювати в окремому терміналі.
4. Відкрийте `index.html` через Live Preview у Chrome або Edge: форма пошуку, у Console немає помилок. Поки Т4 не виконано, «Знайти» нічого не робить.
5. Запустіть `npm run check`: на початку — багато помилок типів і «Тести: пройдено 2 з 40». Прочитайте `README.md` starter-проєкту.
6. Змінюються лише `src/types.ts`, `src/api.ts`, `src/state.ts` і `src/app.ts`. Решта файлів — у тому числі `check/` — не змінюється в жодному коміті.

5.2. Правила роботи з AI-агентом

- **Дослідження (§4).** «Мій прогноз» пишете самі, без AI-агента й до запуску.
- **Т1–Т3 — власноруч.** AI-агент може пояснити документацію TVmaze, помилку компілятора, тест чи поняття TypeScript, але не пише код у `src/`. Приклад запиту в режимі без редагування (ask / plan):

  > Поясни цю помилку компілятора TypeScript. Нижче — повідомлення, рядок з check/types.check.ts разом із коментарем-правилом і мій тип. Не пиши виправлений код: назви, що в моєму типі не так, і де про це прочитати в TypeScript Handbook.

- **Т4 — чотири кроки.**
  1. **Чернетка AI.** Перемкніть AI-агента в IDE в режим, у якому він сам змінює файли проєкту (зазвичай — Agent). Закрийте в редакторі вкладки з файлами `check/`: багато агентів додають відкриті файли до контексту. Вставте в чат **лише** запит Т4 з ТЗ, слово в слово, без власних уточнень.
  2. **Коміт чернетки.** Нічого не виправляючи, закомітьте все, що змінив агент: `T4: AI draft`. Комітьте, навіть якщо чернетка не працює. Якщо агент змінив інші файли, крім `src/app.ts`, перед комітом поверніть їх командою `git restore -- . ':!src/app.ts'` і запишіть це в підсумок звіту.
  3. **Перевірка.** Запустіть `npm run check`. Для кожного тесту чи помилки типів, що не пройшли, з'ясуйте, яке правило Т4 порушено і чому. Правило Т4.5 перевірте в браузері. Перегляньте diff чернетки на обмеження ТЗ, §5: `any`, `@ts-ignore`, а також `as` і `!` — чи справді вони потрібні.
  4. **Виправлення власноруч.** AI може пояснити тест чи помилку, але **не пише** код виправлення. Коли `npm run check` не показує жодної помилки, а чек-лист Network проходить, закомітьте виправлення одним комітом: `T4: fixes`.

  Якщо чернетка з першого разу пройшла всі перевірки, коміту виправлень немає, а в підсумку звіту вкажіть, як ви її перевірили.

У звіті вкажіть назву та версію агента, модель і дату роботи.

5.3. Коміти

Історія Git лабораторної:

1. `Starter` — starter-проєкт без змін;
2. `T1: types`;
3. `T2: api`;
4. `T3: state`;
5. `T4: AI draft` — чернетка агента без змін;
6. `T4: fixes` — якщо чернетка не пройшла перевірку.

Комітити можна й частіше, але кожне завдання закінчується комітом з такою назвою. На GitHub коміт `T4: fixes` показує рівно те, що ви змінили в чернетці AI: на захисті ви пояснюєте за ним, що в чернетці було не так. `node_modules/` і `dist/` не комітяться, а публікувати застосунок не потрібно: викладач запускає `npm run check` у вашому репозиторії.

6. Вимоги до звіту

Тема листа:

```
[Веб] ЛР-11-12: REST API і TypeScript
```

У листі вкажіть репозиторій:

```
GitHub repository: https://github.com/<username>/<repository>
```

Звіт повинен містити:

1. таблицю експериментів §4 (E1–E6);
2. знімок екрана `npm run check` з підсумком «Типи: без помилок · Тести: пройдено 40 з 40»;
3. чек-лист Network Т3 (N1–N4) зі знімками екрана;
4. назву та версію AI-агента, модель, дату роботи;
5. підсумок виконаної роботи: коротко, що ви написали в Т1–Т3, що було не так у чернетці Т4 і що ви виправили; якщо агент змінював файли, які змінювати не можна, — які саме.

7. Усний захист

Формат здачі передбачає усну співбесіду за результатами практичної реалізації. Питання нижче — типові питання співбесіди frontend-розробника з тем цієї роботи. Відповідайте на прикладах власного коду ShowFinder, експериментів §4 і панелі Network. Викладач може попросити показати відповідь у коді чи DevTools або змінити код наживо, без AI. Як оцінюється захист — §8.

Питання для підготовки:

Fetch API і Promise

- Що повертає `fetch()`? Коли його проміс відхиляється, а коли виконується навіть із помилкою сервера?
- Чим `response.ok` відрізняється від `response.status`?
- Чому `response.json()` теж повертає проміс? Що буде, якщо сервер відповів HTML?
- Що повертає async-функція і що стається з винятком, кинутим усередині неї?
- Що означає `Uncaught (in promise)` і як ваш код цього уникає?
- Чим `async`/`await` відрізняється від ланцюжка `then`/`catch`?

HTTP і REST

- Що таке REST API? Що означає ідемпотентність методу GET?
- Що означають статуси 200, 404, 429, 500 і 503? Що з цього бачить користувач ShowFinder?
- Для чого заголовки `Content-Type` і `Cache-Control`?
- Чому параметри запиту треба кодувати? Що сталося б із «law & order» без кодування?

CORS

- Що таке Same-Origin Policy і з чого складається origin?
- Хто забезпечує CORS — браузер чи сервер? Що означає `Access-Control-Allow-Origin: *`?
- Чому адреса відкривається у вкладці, а `fetch` до неї з іншої сторінки — ні?

Стани інтерфейсу

- Які стани має сторінка, що завантажує дані? Чим порожній результат відрізняється від помилки — для коду й для користувача?
- Навіщо рядку статусу `role="status"`?
- Що станеться, якщо користувач запустить другий пошук, поки перший ще триває? Як цьому запобігти?

Типи TypeScript

- Чим `interface` відрізняється від `type`?
- Що таке структурна типізація? Чому об'єкт з API із зайвими полями підходить під `Show`?
- Що таке висновування типів (type inference)? Коли тип варто писати явно?
- Як TypeScript працює з `null` під `strictNullChecks`? Як ви описали поля, що можуть бути `null`?
- Навіщо `readonly`?

Union types, narrowing і type guards

- Що таке union type і discriminated union? Чому `ViewState` — union, а не один об'єкт з необов'язковими полями?
- Якими способами TypeScript звужує тип? Наведіть приклади з вашого коду.
- Що таке type guard (`value is Show`)? Що саме гарантує ваш `isShow`, а чого — ні?
- Чим `unknown` відрізняється від `any`? Чому `getJson` повертає `unknown`?
- Навіщо змінна типу `never` у гілці `default`?

- Чому в strict-режимі змінна в `catch (err)` має тип unknown? Чому `instanceof TypeError` — не надто надійна ознака збою мережі?

Компілятор

- Що робить `tsc`? Що лишається від типів після компіляції?
- Що вмикає `strict`? Які помилки зникли в експерименті E5?
- Навіщо `import type`?
- Чим `@ts-expect-error` відрізняється від `@ts-ignore`? Чому в `src/` обидва заборонені?
- Чим перевірка типів відрізняється від тестів? Що в T1–T4 перевіряє одне, а що — інше?

AI-асистенти в розробці

- Як ваші типи обмежили чернетку AI-агента в T4? Що з помилок агента знайшов компілятор, а що — тести?
- Що було не так у чернетці T4 і як ви це виявили?
- Чому агенту не можна відкривати й змінювати `check/`?

8. Оцінювання

Робота займає два слоти програми — ЛР 11 і ЛР 12, тож за неї — **4 бали** з трьох складників і **1 додатковий бал** за вчасну здачу.

Складник	Бали	Як оцінюється
Самоперевірка	1	`npm run check` у вашому репозиторії: «Типи: без помилок» і всі тести пройдено — 1; без помилок типів і пройдено щонайменше 80 % тестів — 0,5; інакше — 0
Процес	1	коміти за §5.3 і всі пункти звіту §6 — 1; бракує одного — 0,5; більше — 0
Захист	2	питання про власний код — до 1; два питання з різних груп §7 — по 0,5 за повну відповідь
Додатковий бал	+1	лист зі звітом надіслано до **19.10.2026** включно